

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ITMO University

Лабораторная работа 2

По дисциплине Администрирование платформ на ОС Linux

Тема работы Nginx

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

ВВЕДЕНИЕ

Цель работы: развернуть Nginx для веб-приложения и применить бестпрактис в конфигурации.

Задачи: в ходе выполнения лабораторной работы необходимо:

1. развернуть веб-сервер Nginx для работы с веб-приложением;
2. настроить проксирование запросов к backend-серверам;
3. реализовать балансировку нагрузки между несколькими экземплярами приложения;
4. проверить работоспособность веб-приложения и корректность распределения запросов;
5. выполнить моделирование DoS-атаки типа Slowloris;
6. проанализировать влияние атаки на работу сервера, состояние соединений и логи Nginx;
7. настроить защиту от Slowloris-атаки с помощью параметров конфигурации Nginx;
8. выполнить повторную проверку устойчивости сервера после применения защитных механизмов.

Подготовка и развертывание веб-сервера.

В соответствии с заданием предварительно были установлены необходимые компоненты (Node.js, npm, Git и веб-сервер Nginx). После была создана рабочая директория, настроены права доступа, выполнено клонирование репозитория LibreSpeed и установка зависимостей с помощью npm (Рисунок 1).

```
root@rekunid:~# sudo mkdir -p /opt/librespeed
root@rekunid:~# sudo chown $USER:$USER /opt/librespeed
root@rekunid:~# cd /opt/librespeed
root@rekunid:/opt/librespeed# git clone -b node https://github.com/librespeed/speedtest.git
Cloning into 'speedtest'...
remote: Enumerating objects: 2426, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 2426 (delta 15), reused 10 (delta 10), pack-reused 2400 (from 3)
Receiving objects: 100% (2426/2426), 46.67 MiB | 23.14 MiB/s, done.
Resolving deltas: 100% (1425/1425), done.
root@rekunid:/opt/librespeed# cd speedtest/src
root@rekunid:/opt/librespeed/speedtest/src# npm install
```

Рисунок 1 – Клонирование проекта LibreSpeed и установка зависимостей

Далее в исходный код backend-приложения был добавлен маршрут /whoami, позволяющий возвращать порт и идентификатор процесса для проверки балансировки нагрузки, что требовалось по заданию. После этого были запущены несколько экземпляры приложения на разных портах с использованием Node.js (Рисунок 2).

```
root@rekunid:/opt/librespeed/speedtest/src# node SpeedTest.js 8889 &
[1] 10071
root@rekunid:/opt/librespeed/speedtest/src# Speedtest Server is up and running on 8889!
node SpeedTest.js 8888 &
[2] 10081
root@rekunid:/opt/librespeed/speedtest/src# Speedtest Server is up and running on 8888!
node SpeedTest.js 8890 &
[3] 10091
root@rekunid:/opt/librespeed/speedtest/src# Speedtest Server is up and running on 8890!
```

Рисунок 2 – Запуск экземпляров backend-приложения на портах 8888, 8889 и 8890В

В результате в консоли отображаются сообщения о том, что каждый сервер успешно запущен и работает на соответствующем порту. Это подтверждает корректную работу backend-приложения и его готовность к дальнейшей настройке веб-сервера Nginx и организации балансировки нагрузки.

Написание конфигурации

Далее была выполнена настройка веб-сервера Nginx – создан конфигурационный файл (рисунок 3 и рисунок 4). Также был указан домен DuckDNS, корневая директория сайта /opt/librespeed/speedtest/src/public, отключено отображение версии сервера и добавлены заголовки безопасности. В соответствии с заданием были настроены ограничения на методы (разрешены только GET и POST), защита от переполнения буфера с

помощью таймаутов и размеров буферов, а также реализована блокировка доступа по IP-адресу и фильтрация нежелательных User-Agent (ботов и менеджеров загрузки).

```
GNU nano 7.2 /etc/nginx/sites-available/librespeed.conf
upstream librespeed_backend {
    server 127.0.0.1:8888;
    server 127.0.0.1:8889;
    server 127.0.0.1:8890;
}

server {
    listen 80;
    listen [::]:80;
    server_name garriet.duckdns.org;

    root /opt/librespeed/speedtest/src/public;
    index index.html;

    server_tokens off;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy "default-src 'self' 'unsafe-inline' 'unsafe-eval' data: blob;"; always;

    client_header_timeout 10s;
    client_body_timeout 10s;
    keepalive_timeout 15s;
    send_timeout 10s;
    client_body_buffer_size 16k;
    client_header_buffer_size 1k;
    large_client_header_buffers 2 1k;

    if ($host ~* "^[0-9.]+$") {
        return 444;
    }

    if ($http_user_agent ~* "(msnbot|scrapbot|wget|curl|aria2|axel|httpclient|python-requests|libwww-perl)") {
        return 403;
    }

    location / {
        limit_except GET POST {
            deny all;
        }

        proxy_pass http://librespeed_backend;
    }
}
```

Рисунок 3 – Конфигурационный файл Nginx

```
proxy_pass http://librespeed_backend;
proxy_http_version 1.1;

proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}

location /whoami {
    limit_except GET POST {
        deny all;
    }

    proxy_pass http://librespeed_backend/whoami;
    proxy_http_version 1.1;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    return 444;
}
```

Рисунок 4 – Конфигурационный файл Nginx

После завершения настройки была выполнена проверка конфигурации с помощью команды `nginx -t`, в результате чего было получено сообщение об отсутствии ошибок и корректности синтаксиса (Рисунок 5).

```
root@rekunid:/opt/librespeed/speedtest/src# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Рисунок 5 – Проверка корректности конфигурации Nginx

Установка certbot

Далее была выполнена установка Certbot для получения SSL-сертификата (Рисунок 6). Для этого были обновлены пакеты системы, установлен snapd и через него установлен Certbot с последующим созданием символической ссылки для удобного использования. В данной работе был выбран способ получения сертификата с использованием плагина `--nginx`, что позволяет автоматически настроить веб-сервер. В отличие от альтернативного варианта с использованием python-плагина (`webroot` или `standalone`), данный способ является более удобным, так как не требует дополнительной ручной настройки конфигурации и временной остановки сервера.

```
root@rekunid:/opt/librespeed/speedtest/src# apt update
apt install -y snapd
snap install core
snap refresh core
snap install --classic certbot
ln -s /snap/bin/certbot /usr/bin/certbot
Hit:1 http://mirror.selectel.ru/ubuntu noble InRelease
Hit:2 http://mirror.selectel.ru/ubuntu noble-updates InRelease
Hit:3 http://mirror.selectel.ru/ubuntu noble-backports InRelease
Hit:4 https://mirror.selectel.ru/3rd-party/cloud-init-deb/noble noble InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
```

Рисунок 6 – Установка Certbot и подготовка к получению SSL-сертификата

После этого был выполнен запрос сертификата для домена `garriet.duckdns.org` (Рисунок 7). В результате Certbot автоматически выполнил проверку домена, получил сертификат и встроил его в конфигурацию Nginx. Это позволило сразу включить поддержку HTTPS без дополнительного редактирования конфигурационных файлов.

```
root@rekunid:/opt/librespeed/speedtest/src# certbot --nginx -d garriet.duckdns.org
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address or hit Enter to skip.
```

Рисунок 7 – Получение сертификата с автоматической настройкой

После выполнения запроса сертификата было получено сообщение об успешной регистрации аккаунта и начале выпуска SSL-сертификата для

домена `garriet.duckdns.org`. На рисунке 8 представлено подтверждение регистрации и процесс запроса сертификата, что свидетельствует о корректной настройке Certbot.

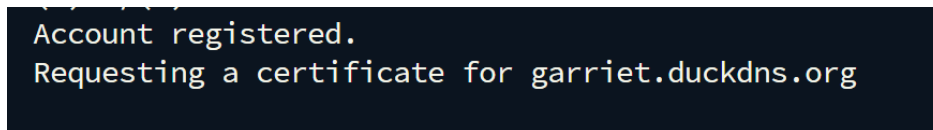


Рисунок 8 – Подтверждение регистрации и запрос SSL-сертификата для домена

После успешной настройки HTTPS была выполнена проверка работы веб-приложения через браузер. На рисунке 9 представлен интерфейс LibreSpeed, открытый по доменному имени, что подтверждает корректную работу веб-сервера и доступность приложения по защищённому протоколу HTTPS.

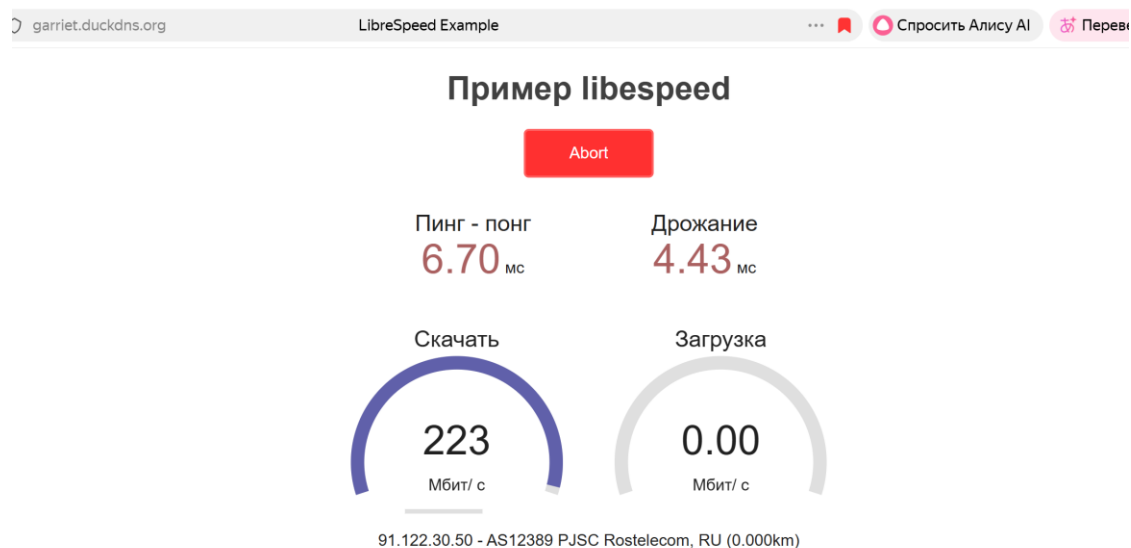


Рисунок 9 – Отображение веб-интерфейса LibreSpeed в браузере

Далее была выполнена проверка балансировки нагрузки между backend-серверами. На рисунке 9 представлены результаты выполнения команды `curl` так, ответы приходят с разных портов (8888, 8889, 8890).

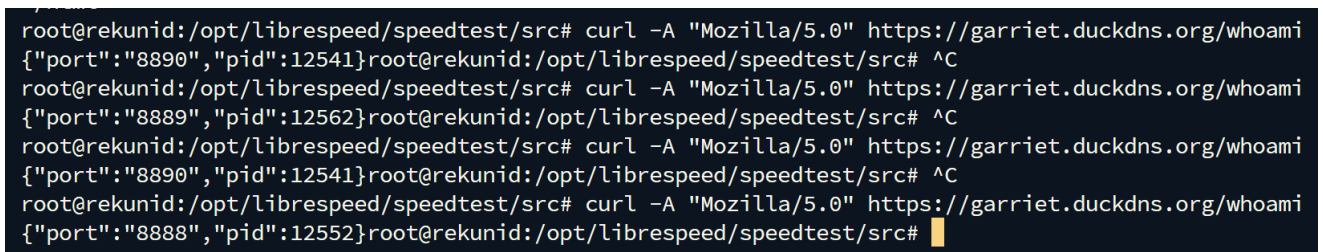
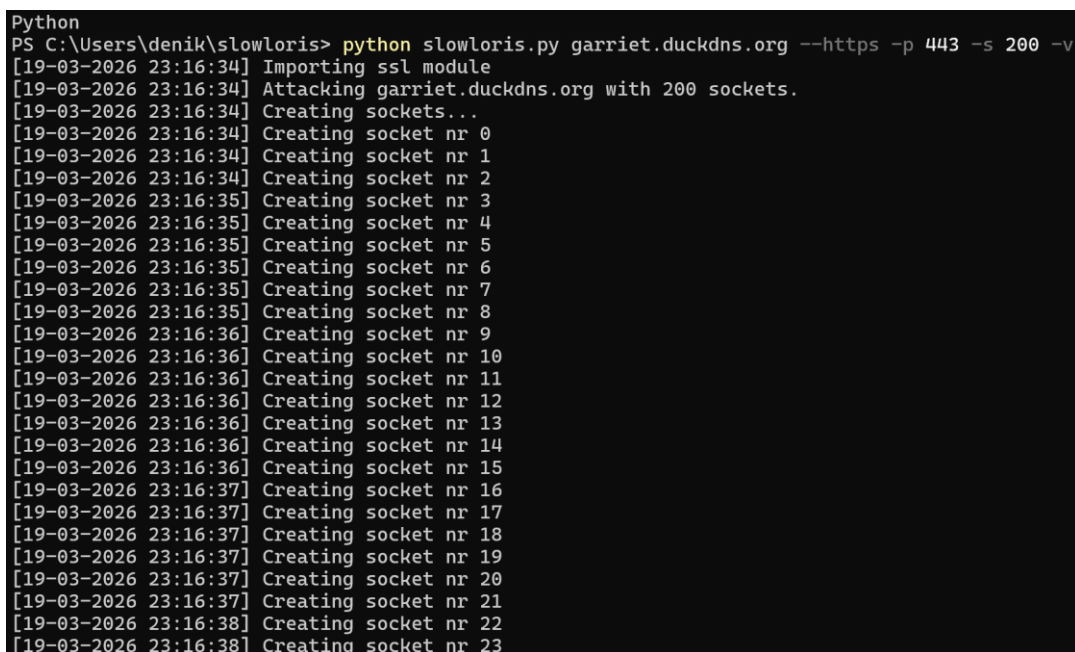


Рисунок 9 – Проверка балансировки нагрузки по эндпоинту `/whoami`

Организация DOS атаки

В рамках лабораторной работы была проведена DoS-атака типа Slowloris на собственный веб-сервер. Для этого использовался скрипт `slowloris.py`, который запускался с указанием доменного имени, протокола HTTPS и порта 443.

На рисунке 10 представлено выполнение скрипта атаки, в ходе которого происходит создание большого количества соединений с сервером. Скрипт последовательно открывает соединения и удерживает их, имитируя медленную передачу данных, что приводит к занятию ресурсов сервера.

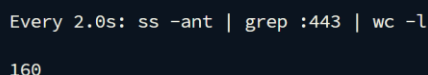


```
Python
PS C:\Users\denik\slowloris> python slowloris.py garriet.duckdns.org --https -p 443 -s 200 -v
[19-03-2026 23:16:34] Importing ssl module
[19-03-2026 23:16:34] Attacking garriet.duckdns.org with 200 sockets.
[19-03-2026 23:16:34] Creating sockets...
[19-03-2026 23:16:34] Creating socket nr 0
[19-03-2026 23:16:34] Creating socket nr 1
[19-03-2026 23:16:34] Creating socket nr 2
[19-03-2026 23:16:35] Creating socket nr 3
[19-03-2026 23:16:35] Creating socket nr 4
[19-03-2026 23:16:35] Creating socket nr 5
[19-03-2026 23:16:35] Creating socket nr 6
[19-03-2026 23:16:35] Creating socket nr 7
[19-03-2026 23:16:35] Creating socket nr 8
[19-03-2026 23:16:36] Creating socket nr 9
[19-03-2026 23:16:36] Creating socket nr 10
[19-03-2026 23:16:36] Creating socket nr 11
[19-03-2026 23:16:36] Creating socket nr 12
[19-03-2026 23:16:36] Creating socket nr 13
[19-03-2026 23:16:36] Creating socket nr 14
[19-03-2026 23:16:36] Creating socket nr 15
[19-03-2026 23:16:37] Creating socket nr 16
[19-03-2026 23:16:37] Creating socket nr 17
[19-03-2026 23:16:37] Creating socket nr 18
[19-03-2026 23:16:37] Creating socket nr 19
[19-03-2026 23:16:37] Creating socket nr 20
[19-03-2026 23:16:37] Creating socket nr 21
[19-03-2026 23:16:38] Creating socket nr 22
[19-03-2026 23:16:38] Creating socket nr 23
```

Рисунок 10 — Запуск и выполнение скрипта Slowloris

Для оценки воздействия атаки на сервер выполнялся мониторинг количества активных соединений с помощью `ss -ant | grep :443 | wc -l`.

На рисунке 11 показан результат выполнения данной команды, из которого видно, что количество соединений достигает значения около 160, что свидетельствует о высокой нагрузке на сервер.



```
Every 2.0s: ss -ant | grep :443 | wc -l
160
```

Рисунок 11 — Количество активных соединений при атаке

Также проводился анализ логов веб-сервера Nginx с использованием команды `tail -f /var/log/nginx/error.log`

На рисунке 12 представлены сообщения из логов сервера. В них наблюдаются ошибки, связанные с обработкой запросов и работой upstream-сервера, что указывает на нестабильность работы сервера в условиях атаки. Таким образом, можно сделать вывод, что без применения защитных механизмов сервер подвержен Slowloris-атаке, так как допускает удержание большого количества одновременных соединений.

```
root@rekunid:/opt/librespeed/speedtest/src# tail -f /var/log/nginx/error.log
2026/03/19 18:37:33 [error] 12525#12525: *266 connect() failed (111: Connection refused) while connecting to upstream, client: 91.231.89.124, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8890/", host: "garriet.duckdns.org"
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8889/", host: "garriet.duckdns.org"
2026/03/19 18:50:39 [error] 12525#12525: *318 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6515349767872144 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
2026/03/19 18:50:39 [error] 12525#12525: *319 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.4344463970622807 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
2026/03/19 18:50:39 [error] 12525#12525: *320 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.2690436250655549 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
2026/03/19 18:50:42 [error] 12525#12525: *321 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.06521788004849782 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
2026/03/19 18:50:42 [error] 12525#12525: *322 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6599160904837853 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
2026/03/19 18:50:45 [error] 12525#12525: *323 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6231156776650413 HTTP/1.1", host: "garriet.duckdns.org", referrer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"
```

Рисунок 12 — Логи веб-сервера Nginx во время атаки

Для более детального анализа состояния соединений использовалась команда `ss -ant | grep :443`

На рисунке 13 представлен вывод данной команды. Видно, что большое количество соединений находится в состоянии FIN-WAIT-2 и TIME-WAIT. Это означает, что сервер активно закрывает соединения, инициированные во время атаки.

```

FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26924
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38038
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39988
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39894
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38074
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26942
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39986
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38064
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39871
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26948
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39881
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26923
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39914
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39912
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38031
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38030
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26944
TIME-WAIT 0 0 192.168.0.19:443 216.73.216.123:38399
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26954
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38061
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38057
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39875
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38087
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39987
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39884
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38077
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38033
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38010
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39857
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26946
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38086
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39989
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38056
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38037
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38039
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39873
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39854
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38058
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38026
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39898
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39885
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:39870
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:26929
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38067
FIN-WAIT-2 0 0 192.168.0.19:443 91.122.30.50:38034
LISTEN 0 511 [::]:443 [::]:*
root@rekunid: /opt/librespeed/speedtest/src#

```

Рисунок 13 — Состояние TCP-соединений на сервере во время атаки

Защита от Slowloris

После организации атаки были добавлены защитные параметры в конфигурацию, направленные на ограничение времени ожидания и уменьшение ресурсов, выделяемых на одно соединение. На рисунке 14 показаны установленные значения таймаутов и буферов, а также включён параметр `reset_timedout_connection`, который позволяет принудительно закрывать «зависшие» соединения.

```

client_header_timeout 5s;
client_body_timeout 5s;
keepalive_timeout 5s;
send_timeout 5s;
client_body_buffer_size 8k;
client_header_buffer_size 1k;
large_client_header_buffers 2 1k;
reset_timedout_connection on;

```

Рисунок 14 — Настройки защиты от Slowloris в конфигурации Nginx

Далее была проведена проверка количества активных соединений с помощью команды `ss -ant | grep :443 | wc -l`. Как видно на рисунке 15, количество соединений снизилось до 22, что значительно меньше по сравнению с этапом атаки без защиты.

```
Every 2.0s: ss -ant | grep :443 | wc -l  
22
```

Рисунок 15 — Количество активных соединений после применения защиты

На рисунке 16 представлены логи сервера Nginx, в которых фиксируются ошибки вида `client intended to send too large body`, а также `connection refused`. Это говорит о том, что сервер начал активно ограничивать некорректные запросы и отказывать в обслуживании подозрительным подключениям.

```
root@ekund:/opt/tiberspeed/speedtest/src# cat -v /var/log/nginx/error.log  
2026/03/19 18:37:33 [error] 12525#12525: *266 connect() failed (111: Connection refused) while connecting to upstream, client: 91.231.89.124, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"  
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"  
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"  
2026/03/19 18:40:53 [error] 12525#12525: *270 connect() failed (111: Connection refused) while connecting to upstream, client: 91.196.152.165, server: garriet.duckdns.org, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8888/", host: "garriet.duckdns.org"  
2026/03/19 18:50:39 [error] 12525#12525: *318 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6515349767872144 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
2026/03/19 18:50:39 [error] 12525#12525: *319 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.4344463970622807 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
2026/03/19 18:50:39 [error] 12525#12525: *320 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.2690436250655549 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
2026/03/19 18:50:42 [error] 12525#12525: *321 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.06521788004849782 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
2026/03/19 18:50:42 [error] 12525#12525: *322 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6599160904837853 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
2026/03/19 18:50:45 [error] 12525#12525: *323 client intended to send too large body: 20971520 bytes, client: 91.122.30.50, server: garriet.duckdns.org, request: "POST /empty?r=0.6231156776650413 HTTP/1.1", host: "garriet.duckdns.org", referer: "https://garriet.duckdns.org/speedtest_worker.js?r=0.3058507218091294"  
^C
```

Рисунок 12 — Логи сервера Nginx во время атаки после внедрения защиты

ВЫВОДЫ

В ходе выполнения лабораторной работы была развернута веб-инфраструктура с использованием Nginx и нескольких бэкенд-серверов, настроена балансировка нагрузки и проверена её работоспособность. Далее была реализована и исследована DoS-атака типа Slowloris, в результате которой было показано, что сервер уязвим не по нагрузке на процессор, а по количеству удерживаемых соединений. Был проведён анализ состояния соединений и логов сервера, что позволило выявить характер поведения системы во время атаки.

После этого были применены защитные механизмы на уровне Nginx, включающие настройку таймаутов, ограничение буферов и принудительное закрытие зависших соединений. Повторное тестирование показало значительное снижение количества активных соединений и повышение устойчивости сервера к атаке. Таким образом, было продемонстрировано, что эффективная защита достигается не увеличением ресурсов, а правильной настройкой сетевых параметров и контролем соединений на уровне приложения.

Ответы на вопросы.

1. **Зачем нужна балансировка?** Балансировка нужна для распределения нагрузки между несколькими серверами, чтобы запросы не обрабатывались одним бэкендом, а равномерно делились между всеми. Это повышает производительность системы и её устойчивость, потому что если один сервер перестаёт работать, остальные продолжают обслуживать пользователей.
2. **Убейте процессы бэкендов и откройте сайт. Что произошло и как вы это объясните?** В этом случае сайт перестаёт открываться или появляется ошибка, например 502 Bad Gateway, потому что Nginx выступает как прокси и пересылает запросы на бэкенды. Если бэкенды не запущены, ему просто некуда отправлять запрос, поэтому пользователь получает ошибку.
3. **Запись `Server.listen(PORT, '127.0.0.1')` означает, что мы слушаем только с адреса 127.0.0.1, почему не указать 0.0.0.0?** Адрес 127.0.0.1 означает, что сервер доступен только локально, с самого хоста, и это делается для безопасности, чтобы к бэкенду нельзя было подключиться напрямую извне. Если указать 0.0.0.0, сервис станет доступен с любых интерфейсов, включая внешний, и тогда можно будет обойти Nginx и его защиту.
4. **Почему slowloris-атака бьёт по пулам соединений, а не CPU?** Такая атака основана на удержании большого количества открытых соединений, при этом данные отправляются очень медленно, поэтому процессор практически не нагружается. Основной расход идёт на соединения и ресурсы сети, потому что сервер держит открытые сокеты и ожидает завершения запросов.
5. **Почему использовали Nginx в седьмом уровне?** Потому что Nginx работает на уровне приложений и понимает HTTP-протокол, за счёт этого можно управлять заголовками, методами, таймаутами и фильтровать запросы. Это даёт возможность эффективно защищаться от атак, связанных с особенностями HTTP.
6. **Почему не отразить slowloris-атаку просто увеличив количество воркеров?** Увеличение воркеров не решает проблему, а лишь временно откладывает её, потому что атака всё равно займёт все доступные соединения и ресурсы. В итоге сервер снова станет недоступен, поэтому важно не увеличивать ресурсы, а ограничивать время соединений и закрывать подозрительные запросы.